LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

CVE-2026-11822 SQLite before 3.53.2 Memory Corruption in FTS5 Extension

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-15
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

Published: 2026-06-15 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary

CVE-2026-11822 affects SQLite versions before 3.53.2 and centers on memory-corruption conditions in the FTS5 full-text search extension. Public CVE enrichment describes two concrete fault paths reachable when a vulnerable application executes an FTS5 `MATCH` query against an attacker-supplied database containing malformed FTS5 page data: an out-of-bounds read in `fts5LeafSeek()` and a heap buffer overflow write in `fts5ChunkIterate()` caused by an integer underflow on a crafted continuation page. The resulting outcomes range from process crashes and memory exhaustion to potential arbitrary code execution within the context of the vulnerable application.

The exposure profile is narrower than a typical network-service RCE because exploitation requires an application to ingest an untrusted SQLite database and to exercise the FTS5 search path, usually through user interaction, background indexing, content inspection, or application-level query execution. That said, the issue remains materially important for defenders because SQLite is deeply embedded across desktop software, mobile apps, endpoint tools, browsers, collaboration clients, local caches, and internal utilities. Where FTS5 is compiled in and untrusted databases can be opened or synchronized into local workflows, the bug creates a plausible malicious-file execution path.

Upstream SQLite release material confirms that 3.53.2 fixes problems in the 3.53.0 line and linked check-ins explicitly state they fix a potential FTS5 buffer overwrite when processing corrupt records, but the upstream release notes are sparse on exploit mechanics. Downstream CVE tracking fills in the technical detail and currently rates the issue High severity. At assessment time, reviewed public sources did not provide authoritative evidence of Known Exploited Vulnerabilities (KEV) inclusion or broad in-the-wild exploitation. Defenders should therefore prioritize version remediation, review where FTS5-enabled SQLite is exposed to attacker-controlled content, and hunt for crash telemetry, suspicious database ingestion, and application workflows that execute FTS5 queries over untrusted data.

2. Intelligence Requirement

This report addresses the following intelligence requirement: what does CVE-2026-11822 mean for defenders using SQLite with FTS5 enabled, which environments are materially exposed, how should public severity be interpreted, and what remediation, detection, and hunting actions should be prioritized now?

Question	Assessment
What is the flaw?	Memory-corruption vulnerabilities in SQLite FTS5 prior to 3.53.2, including an out-of-bounds read in `fts5LeafSeek()` and a heap buffer overflow write in `fts5ChunkIterate()` when parsing malformed FTS5 page data.
What is the primary impact?	Process crash, memory exhaustion, and plausible arbitrary code execution in the context of the application processing the crafted database.
Which products are affected?	SQLite before 3.53.2 where the FTS5 extension is present and reachable.
What is the trigger condition?	A vulnerable application executes an FTS5 `MATCH` query against attacker-controlled SQLite data containing malformed FTS5 page structures.
Is this a pure network exploit?	No. Public scoring and descriptions indicate a local or malicious-file style attack path rather than unauthenticated remote exploitation of a listening service.
What fixes the issue?	Upgrade to SQLite 3.53.2 or a vendor-packaged build that backports the same FTS5 fixes.
Is broad exploitation confirmed?	No authoritative broad in-the-wild exploitation evidence was identified in reviewed public sources, and the CVE is not present

	in the CISA KEV catalog snapshot reviewed for this task.
Which environments deserve highest priority?	Applications that open untrusted SQLite databases, sync SQLite content from external parties, or embed FTS5-enabled SQLite for search/indexing across hostile or semi-trusted content boundaries.

3. Source Review and Confidence

Source	Contribution	Confidence
SQLite 3.53.2 release log	Confirms 3.53.2 as the fixing release and establishes upstream remediation timing	High
SQLite branch/trunk check-ins `061febcf41ca` and `4a5ad516ea93`	Primary engineering evidence that SQLite patched a potential FTS5 buffer overwrite tied to corrupt-record handling	High
NVD JSON 2.0 / NVD detail page	Supplies the fullest public technical description, CVSS scoring, CWE mapping, and upstream reference links	High
GitHub Advisory Database entry `GHSA-6qj8-gw6p-hc5p`	Independent downstream consolidation of the same vulnerable behavior and impact language	Medium
Microsoft Security Update Guide syndication entry	Workflow trigger confirming publication visibility in monitored intake feeds	Medium
CISA KEV catalog snapshot	Negative-control check for known exploitation status at time of review	High

Confidence is high that SQLite versions before 3.53.2 contain an FTS5 parsing flaw fixed in the 3.53.2 timeframe, because NVD references align with upstream SQLite release and check-in material. Confidence is medium-high for detailed exploitation mechanics because the richest public technical narrative currently comes from downstream enrichment rather than a full upstream security advisory naming CVE-2026-11822 directly. Upstream SQLite release notes remain terse, so defenders should treat NVD and linked advisory consolidation as useful technical context while still privileging SQLite's 3.53.2 remediation status as the authoritative fix point.

One analytic caveat matters: public scoring is severity-rich but context-light. NVD currently scores the issue CVSS 3.1 7.8 High and CVSS 4.0 8.5 High, yet the attack path still depends on a vulnerable application processing attacker-controlled SQLite content and invoking FTS5 query logic. Defenders should prioritize environments where malicious files or synchronized data can traverse meaningful trust boundaries instead of assuming every SQLite presence is equally exposed.

4.1 Vulnerability metadata

Field	Value
CVE	CVE-2026-11822
Product family	SQLite
Affected component	FTS5 full-text search extension
Vulnerability class	Memory corruption in malformed FTS5 page parsing
Weakness	CWE-122 Heap-based Buffer Overflow
Public severity	CVSS 3.1 7.8 High; CVSS 4.0 8.5 High

Affected versions	SQLite before 3.53.2
Fixed version	SQLite 3.53.2
NVD publication	2026-06-09
SQLite fix evidence	Upstream release 3.53.2 (2026-06-03) and linked trunk/branch check-ins dated 2026-05-11
Exploitation status	No authoritative KEV inclusion or broad in-the-wild exploitation reporting identified during this review
Delivery style	Crafted database / malicious content processing path

4.2 Flaw mechanics

Public CVE enrichment states that an attacker can supply a crafted SQLite database containing malformed FTS5 page data and then trigger vulnerable code paths when the target application executes an FTS5 `MATCH` query. Two named functions are called out in public descriptions:

- `fts5LeafSeek()` can perform an out-of-bounds read because an attacker influences a loop bound while malformed page content is being processed.
- `fts5ChunkIterate()` can perform a heap buffer overflow write when a crafted continuation page causes an integer underflow during chunk iteration.

The public impact language spans process crash, memory exhaustion, and arbitrary code execution. From a defender's perspective, the most realistic near-term outcomes are denial of service and application instability, but the heap write condition is serious enough that environments processing attacker-controlled content should not dismiss the code-execution possibility.

The upstream fix evidence is concise but important. The SQLite branch and trunk check-ins linked by NVD both state: `Fix potential buffer overwrite that could occur in fts5 when processing corrupt records.` SQLite's 3.53.2 release notes do not enumerate CVE-2026-11822 directly, but they establish 3.53.2 as the patch level containing the repair.

4.3 Exposure profile

Exposure pattern	Why it matters	Priority
Desktop, server, or embedded applications that open untrusted `.db` / `.sqlite` files and support search features	A malicious database can be delivered through file exchange, downloads, archives, sync tools, or user-submitted content	Immediate
Products that bundle SQLite with FTS5 enabled for local indexing, search, note-taking, messaging, or content catalogs	FTS5 parsing becomes reachable as part of normal indexing/search behavior rather than an obscure code path	Immediate
Internal automation that imports third-party SQLite datasets before querying them	Semi-trusted ingest pipelines can become exposure points even without direct internet-facing access	High
Estates using SQLite without FTS5 or without opening external databases	Reduces or removes the reachable vulnerable surface described publicly	Reduced
Systems already on SQLite 3.53.2 or vendor-confirmed backports	Remediation removes the known vulnerable path if the package truly carries the fix	Reduced

4.4 Defensive significance and ATT&CK framing

MITRE ATT&CK mapping is conditional because CVE-2026-11822 is a vulnerable-parser condition rather than an actor-specific intrusion set. The best-fit mappings should be used to frame telemetry and defensive implications, not to imply a confirmed campaign sequence.

ATT&CK ID	Technique	Rationale
T1204.002	User Execution: Malicious File	The likely delivery model is a crafted SQLite database file opened or processed by a user or application workflow.
T1203	Exploitation for Client Execution	The malicious database triggers memory corruption inside a vulnerable local application component when FTS5 query logic runs.

The practical takeaway is straightforward: if an adversary can place a malicious SQLite database into a workflow that subsequently executes FTS5 search operations, they may be able to crash the application or potentially achieve code execution in the victim context.

5. Threat Context

CVE-2026-11822 is best treated as a malicious-content and local-processing risk, not a universal internet-facing RCE. The attacker does not simply send one packet to a listening SQLite service. Instead, they need a way to deliver or position a crafted SQLite database so that a vulnerable application opens it and invokes FTS5 logic. That makes the issue especially relevant to desktop software, collaboration tools, content-management workflows, sync agents, forensic utilities, offline caches, and internal applications that import SQLite-backed datasets from third parties.

The absence of KEV inclusion or authoritative campaign reporting lowers concern for indiscriminate opportunistic exploitation at the time of this review. However, defenders should not over-index on that absence. SQLite is one of the most broadly embedded software components in modern computing, and many teams do not have a complete inventory of which applications statically bundle SQLite or whether FTS5 was enabled at build time. A moderately complex malicious-file exploit path can still be operationally significant when the vulnerable library is ubiquitous and silently embedded.

5.1 Representative abuse scenarios

Scenario	Description	Defensive meaning
Malicious project or content bundle	An attacker shares an archive containing a crafted SQLite database that a target application later indexes or searches with FTS5	Treat inbound SQLite files as active content, not inert data
Supply-chain style dataset ingestion	Internal tooling imports partner, customer, or open-source SQLite datasets and runs FTS queries for validation or search	Review ingest pipelines that treat external SQLite data as trusted input
Embedded product exploitation	A desktop or appliance product statically bundles vulnerable SQLite and uses FTS5 for internal search, exposing users who open attacker-controlled workspaces	Patch bundled SQLite copies, not just OS packages
Lateral delivery through collaboration sync	A malicious database is placed into a shared folder or sync channel and later opened by another user's FTS-enabled application	Hunt for unusual SQLite file ingress into collaboration and content-sync paths

5.2 Exploitation-status assessment

Reviewed public sources did not identify CISA KEV inclusion, widespread incident reporting, or named threat-actor campaigns leveraging CVE-2026-11822. The available record is presently dominated by public CVE enrichment and SQLite fix artifacts rather than by exploitation telemetry.

Exposure should therefore be prioritized by workflow risk, not by media volume. Environments that never ingest untrusted SQLite content or do not expose FTS5 are lower priority after validation. Environments that exchange SQLite-backed datasets across trust boundaries, especially where search or indexing is automatic, should treat this as a higher-urgency patching and review item.

6. Detection and Hunting

Detection for CVE-2026-11822 should focus on exposure discovery, vulnerable-version inventory, suspicious SQLite file ingress, and crash telemetry tied to FTS5 parsing. No stable attacker infrastructure or malware-family IOC set was identified in reviewed public sources. This is therefore an exposure-management and malformed-content hunting problem rather than a classic domain/IP/hash blocking problem.

6.1 Detection coverage summary

Signal	Telemetry source	Analytic goal	Coverage caveat
SQLite versions below 3.53.2 or unknown bundled copies	Software inventory, SBOM, package telemetry, application manifests	Identify systems needing patch validation	Static linking and vendor backports can obscure true version state
Applications, binaries, or source trees referencing FTS5 functionality	Secure code review, repo scans, binary strings, SBOM metadata	Determine whether the vulnerable extension is actually present and reachable	Presence of FTS5 does not prove untrusted-input exposure
Crashes referencing SQLite / FTS5 parser paths	App crash logs, endpoint telemetry, debugger traces, WER	Surface potential exploit attempts or stability issues	Many environments do not retain symbol-rich crash logs
New or transferred SQLite database files from untrusted locations	EDR file telemetry, proxy/download logs, collaboration telemetry	Identify likely delivery vectors for malicious content	Benign SQLite file movement is common in developer estates
Automated FTS queries over imported external datasets	Application logs, service traces, ETL job logs	Find workflows where attacker-controlled data becomes queryable	Often requires application-specific telemetry rather than generic EDR

6.2 Sigma - SQLite and FTS-related crash telemetry review

```
title: Potential SQLite FTS5 Crash Activity Requiring CVE-2026-11822 Review
id: b8a1d04e-d440-4f45-9d6f-d8b7e3e4c247
status: experimental
description: Detects Windows application-error telemetry that may indicate SQLite/FTS-related crash
conditions. Intended for triage and hunting, not direct exploit confirmation.
author: Lost Boys Cyber
date: 2026-06-15
references:
  - https://nvd.nist.gov/vuln/detail/CVE-2026-11822
  - https://sqlite.org/releaselog/3_53_2.html
logsource:
  product: windows
  service: application
detection:
  selection_provider:
    Provider_Name: 'Application Error'
  selection_message:
```

```
    Message|contains:
      - 'sqlite'
      - 'fts5'
      - 'fts5LeafSeek'
      - 'fts5ChunkIterate'
  condition: selection_provider and selection_message
falsepositives:
  - Legitimate application instability unrelated to CVE-2026-11822
  - Developer crash testing against instrumented SQLite builds
level: medium
tags:
  - attack.t1203
  - attack.t1204.002
```

6.3 KQL - inventory potentially affected SQLite software

```
DeviceTvmSoftwareInventory
| where SoftwareName has "sqlite"
| project DeviceName, SoftwareName, SoftwareVersion, Vendor
| order by DeviceName asc, SoftwareName asc
```

6.4 KQL - hunt for suspicious SQLite file ingress followed by process execution or crashes

```
let lookback = 30d;
let sqliteFiles = DeviceFileEvents
| where Timestamp > ago(lookback)
| where FileName matches regex @"(?i).+\.(db|sqlite|sqlite3)$"
| where FolderPath has_any ("Downloads", "Temp", "AppData", "Desktop", "Mail", "Attachments",
"OneDrive", "Share")
| project FileTimestamp=Timestamp, DeviceId, DeviceName, FileName, FolderPath, SHA1,
InitiatingProcessFileName, InitiatingProcessCommandLine;
let sqliteProcesses = DeviceProcessEvents
| where Timestamp > ago(lookback)
| where ProcessCommandLine has_any (".db", ".sqlite", ".sqlite3", "MATCH ")
| project ProcTimestamp=Timestamp, DeviceId, DeviceName, FileName, ProcessCommandLine,
InitiatingProcessFileName;
sqliteFiles
| join kind=leftouter sqliteProcesses on DeviceId
| where isnull(ProcTimestamp) or ProcTimestamp between (FileTimestamp .. FileTimestamp + 2d)
| order by FileTimestamp desc
```

6.5 SPL - crash and malformed-database triage for SQLite/FTS activity

```
(index=wineventlog OR index=os OR index=app)
("sqlite" OR "fts5" OR "fts5LeafSeek" OR "fts5ChunkIterate" OR "database disk image is malformed")
| stats count values(host) values(source) values(process_name) values(file_path) values(Message) by
sourcetype
| sort - count
```

6.6 Hunt questions

Hypothesis	What to prove or disprove
Some enterprise applications bundle vulnerable SQLite builds that are absent from OS-level package inventory	Inspect application directories, SBOMs, and binary strings for embedded SQLite/FTS5 indicators
FTS5 is enabled in products that process external databases or imported workspaces	Review product documentation, compile flags, source trees, and local test builds
SQLite databases are entering the environment through user downloads, archives, email, or sync channels	Correlate file-ingress telemetry with applications known to use SQLite-backed search or indexing
Crash telemetry already reflects malformed-database handling problems	Review historical application error logs and WER-style telemetry for SQLite/FTS-related fault strings

No stable attacker IOC set was identified from reviewed public sources. Hunting should center on version exposure, file-ingress paths, embedded SQLite copies, and FTS5-enabled workflows that cross trust boundaries.

7. Recommendations

Priority	Owner	Recommendation	Rationale
Critical	Endpoint engineering / product owners	Upgrade SQLite to 3.53.2 or vendor-confirmed backports everywhere it is packaged	Removes the publicly described vulnerable FTS5 parser paths
High	Application owners	Inventory applications that statically embed SQLite and confirm whether FTS5 is compiled and reachable	OS package patching alone may miss bundled copies
High	Security engineering	Treat inbound SQLite databases as active content in email, download, sync, and customer-ingest workflows	The likely exploit path is malicious-file delivery into a vulnerable local parser
High	Detection engineering	Deploy exposure-discovery analytics for SQLite versions, suspicious SQLite file ingress, and FTS-related crashes	Traditional IOC blocking is not the primary defensive control here
Medium	Developers / QA	Add regression tests for malformed SQLite/FTS5 content and isolate untrusted database parsing where feasible	Reduces repeat parser-risk classes and improves future resilience
Medium	Incident response	Preserve suspect SQLite files and crash artifacts for forensic reproduction if a relevant application begins faulting unexpectedly	Crash evidence may be the first sign of exploit attempts or malformed-content targeting

Immediate defensive priorities are straightforward: patch, inventory bundled copies, validate whether FTS5 is enabled, and focus hunts on untrusted SQLite ingestion rather than on generic network scanning. If a vulnerable application automatically imports or searches third-party SQLite data, raise remediation urgency even in the absence of active-exploitation reporting.

1. SQLite Release 3.53.2 On 2026-06-03

- https://sqlite.org/releaselog/3_53_2.html

2. SQLite Release History / changes page

- https://sqlite.org/changes.html

3. SQLite trunk check-in `4a5ad516ea93`

- https://sqlite.org/src/info/4a5ad516ea93

4. SQLite branch-3.53 check-in `061febcf41ca`

- https://sqlite.org/src/info/061febcf41ca

5. NVD CVE Record: CVE-2026-11822

- https://nvd.nist.gov/vuln/detail/CVE-2026-11822

6. NVD JSON 2.0 API entry for CVE-2026-11822

- https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-11822

7. GitHub Advisory Database entry `GHSA-6qj8-gw6p-hc5p`

- https://github.com/advisories/GHSA-6qj8-gw6p-hc5p

8. Microsoft Security Update Guide syndication entry

- https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-11822

9. CISA Known Exploited Vulnerabilities catalog

- https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json